

Задача А. Обои

Сделаем то, что требуется в условии: пройдемся по массиву A , проверяя, делится ли элемент $A[i]$ на k . Если он делится, то к ответу стоит добавить $A[i]/k$, в противном случае стоит добавить $\lfloor A[i]/k \rfloor + 1$.

Задача В. Старый тетрис

Заметим, что каждые два уровня мы будем переходить на следующие сначала тремя тетрисами, а затем двумя тетрисами. Рассмотрим на примере первых четырех уровней, сколько очков мы заработаем. Обозначим через *tetris* количество очков, которые даёт тетрис на нулевом уровне, через *step* — увеличение стоимости тетриса. За первые 2 уровня будут заработаны $3 * tetris + 2 * (tetris + step) = 5 * tetris + 2 * step$ очков. На третьем и четвертом уровне игрок заработает $3 * (tetris + 2 * step) + 2 * (tetris + 3 * step) = 5 * tetris + 12 * step$ очков. Несложно убедиться, что количество очков будет увеличиваться на величину $10 * step$. Отсюда вытекает решение: бинарным поиском найдём количество пар уровней, на которых сумма очков в первый раз станет строго больше X при помощи подсчёта суммы элементов арифметической прогрессии. После этого остаётся проверить, на каком из уровней пары в первый раз сумма стала не меньше X .

Задача С. И снова лабиринт

Найдём клетки, лежащие хотя бы на одном кратчайшем пути. Это делается двумя bfs — от начальной и от конечной клетки. Оставим в графе только эти клетки и найдём точки сочленения — получим клетки, лежащие на любом кратчайшем пути. Установка препятствия в любой из них либо удлинит кратчайший путь, либо вообще приведёт к отсутствию пути.

Эти два случая как-то надо различить. Рассмотрим два различных способа для этого.

В первом способе найдём точки сочленения в исходном графе, но с дополнительным условием: нам нужны только такие точки, которые разрывают все пути от *start* в *finish*. Для этого модифицируем стандартный алгоритм поиска точек сочленения следующим образом. В стандартном алгоритме вершина v является точкой сочленения, если есть такое ребро $v - to$, что ни из какого потомка to нет ребра ни в какого предка v . Здесь же мы добавим дополнительное условие: среди потомков to должен быть *finish*. Тогда при рекурсивной реализации функция *dfs* может возвращать *true*, если посетила вершину *finish*, и *false*, если нет. Запускать *dfs* нужно обязательно из вершины *start*.

Во втором способе воспользуемся системой непересекающихся множеств с отменами, которую будем строить не для свободных, а для занятых клеток. Будем считать, что у нас есть дополнительная область занятых клеток из дополнительного столбца слева и строки снизу, а также область из дополнительного столбца справа и строки сверху. Соседей будем рассматривать не только по стороне, но и по углу. Если при установке препятствия у нас соединятся эти две области, то все пути окажутся перекрыты, а иначе какой-то путь остаётся. Потом делаем отмену этой операции, переходим к следующей клетке, и так далее.

Ещё прямо во время тура жюри нашло и реализовало ещё один способ клеток, разрывающих путь между левой верхней и правой нижней клетками. Способ связан со спецификой задачи. Построим дополнительные стены из занятых клеток слева и снизу поля, а вторую стену — справа и сверху. Рекурсивно закрасим эти две стены в разные цвета (включая примыкающие к ним ячейки). Критерий, что клетка при удалении разорвёт достижимость из левого верхнего угла в правый нижний — если у неё есть два соседа разных цветов. Этот способ применяем два раза — к оригинальному графу (чтобы понять, какие клетки не трогать) и к графу, составленному только из клеток на кратчайших путях.

Корректность первой части алгоритма (первый абзац разбора), возможно, стоит пояснить подробнее. Мы создаём новый граф только из вершин, лежащих хотя бы на одном кратчайшем пути из *start* в *finish*, и в полученном графе находим точки сочленения. Надо доказать два утверждения.

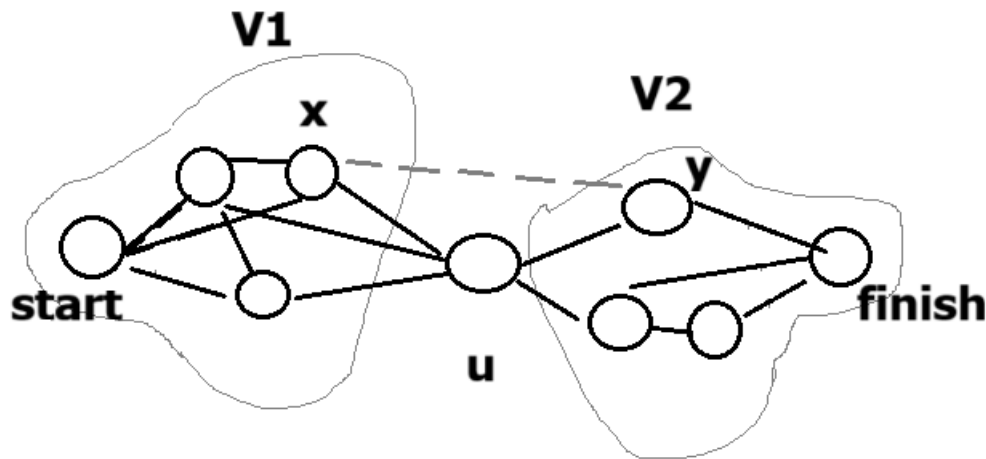
1. Любая точка сочленения удлиняет кратчайший путь в исходном графе либо убирает путь совсем.
2. Других вершин с таким свойством нет.

Докажем утверждение 1. Предположим, что какая-то точка сочленения в новом графе не удлиняет кратчайший путь в исходном графе. Это значит, что при её удалении в новом графе останется какой-то путь из *start* в *finish*. Но раз это всё-таки точка сочленения, значит, при её удалении в новом графе образуется компонента, в которую нельзя попасть ни из *start*, ни из *finish*, то есть лежащая на «ответвлении» от кратчайшего пути. Тогда её вершины не лежат на кратчайших путях из *start* в *finish*. Однако, этого быть не может, так как новый граф был составлен только из вершин на кратчайших путях.

Докажем утверждение 2. Предположим, существует вершина *u* помимо найденных, удлиняющая кратчайший путь в исходном графе. Очевидно, она должна присутствовать и в новом графе. Пусть она не является в нём точкой сочленения. То есть, при её удалении в новом графе остаётся путь из *start* в *finish*, но он становится длиннее (если бы длина не увеличилась, то и в исходном графе она не увеличилась бы). Раз длина увеличилась, значит, в новом графе все кратчайшие пути из *start* в *finish* проходили через вершину *u*. Разделим все оставшиеся вершины на два множества:

- V_1 — вершины, которые на кратчайших путях находились до вершины *u*,
- V_2 — вершины, которые на кратчайших путях находились после вершины *u*.

Понятно, что V_1 и V_2 не пересекаются.



Предположим теперь, что существует какой-то более длинный путь из *start* в *finish*, не проходящий через *u*. Тогда в нём есть ребро между вершиной *x* из V_1 и вершиной *y* из V_2 . Но вершина *x* лежит на каком-то кратчайшем пути из *start* в *finish*, проходящем через *u*, и вершина *y* — тоже. Тогда путь из трёх кусочков: кратчайший путь от *start* до *x*, затем ребро *x-y* и кратчайший путь от *y* до *finish* будет короче, чем кратчайший путь через *u* — пришли к противоречию.

Задача D. Колеблющаяся подпоследовательность

Задача допускает жадное решение. Рассмотрим колеблющуюся последовательность $(a_1, a_2, \dots, a_{k-1}, a_k)$ длины k , тогда добавление к ней любого элемента x порождает колеблющуюся последовательность длины не менее k . Допустим, что $a_{k-1} > a_k$ — возможны две ситуации:

1. $x > a_k$, тогда добавим x после a_k что даёт нам последовательность длины $(k + 1)$: $a_1, a_2, \dots, a_{k-1}, a_k, a_{k+1} = x$
2. $x \leq a_k$, тогда заменим a_k на x (мы можем это сделать в силу того, что $a_{k-1} > a_k \geq x$), и получим последовательность $(a_1, a_2, \dots, a_{k-1}, x)$ той же длины, но с меньшим последним элементом. Предыдущую последовательность можно забыть, так как если некий элемент y можно присоединить к $(a_1, a_2, \dots, a_{k-1}, a_k)$, то $y > a_k \geq x$, то его можно присоединить и к $(a_1, a_2, \dots, a_{k-1}, x)$.

Симметричные рассуждения справедливы для случая $a_{k-1} < a_k$.

Для решения задачи необходимо хранить две подпоследовательности: $(a_1, \dots, a_{k-2} < a_{k-1} > a_k)$ и $(b_1, \dots, b_{k-2} > b_{k-1} < b_k)$, и пробовать присоединять к ним каждый элемент по очереди. После обработки всей исходной последовательности – выбрать наиболее длинную.

Задача Е. Плюсы и минусы

Разобьём выражение на токены. С помощью стека для каждой открывающей скобки определим позицию закрывающей.

Если в начале выражения стоит скобка, то переходим на одну позицию правее соответствующей ей закрывающей, далее идём, поддерживая скобочный баланс. При балансе 0 все знаки меняем на '-', при большем балансе — на '+'. Далее рекурсивно решаем ту же задачу для начальной части выражения в скобках. Пример:

$((2-3)-(5-2))+(2-4)+(3-(4-5))$

Для наглядности выделим первую скобку и соответствующую ей закрывающую квадратными скобками:

$[(2-3)-(5-2)]+(2-4)+(3-(4-5))$

Во всех позициях после закрывающей квадратной скобки все знаки операций внутри скобок заменяем на плюсы, а снаружи скобок — на минусы. Для этого достаточно пройти одним циклом, поддерживая баланс скобок:

$[(2-3)-(5-2)]-(2+4)-(3+(4+5))$

Выражение внутри квадратных скобок — это задача такого же вида, как исходная, поэтому её решаем аналогичным образом. Примечание: конечно, не надо заново находить для каждой открывающей скобки позицию закрывающей, это достаточно сделать один раз.

Задача Ф. Классический тетрис: подсчёт очков

Требовалось выполнить то, что написано в условии, поддерживая количество линий, необходимое для перехода на следующий уровень, количество очков и номер текущего уровня.

Задача Г. Карточная колода

По сути, тасовка карт является простой математической перестановкой. Обозначим как $P(i)$ место, на которое попадает i -я карта после тасовки. Пусть также $P^2(i) = P(P(i))$, $P^3(i) = P(P(P(i)))$, Так как количество мест ограничено n , то для любого i найдется такое t_i , что $P^{t_i}(i) = i$. Таким образом, вся перестановка P разбивается на несколько циклов разной длины. Наименьшее общее кратное длин этих циклов и будет ответом.

Ограничения для задачи подобраны таким образом, что при аккуратной реализации ответ помещается в 64-х битное целое.

P.S. При указанной системе тасовки максимум достигается при 666 картах... Азартные игры-с...

Задача Н. IP v6

Разрезаем входную строку по «:». Должно получиться 8 частей. Если их меньше – то было два двоеточия подряд, и одна из полученных частей пустая. Вставим на её место нужное количество нулей. Выводим части в виде четырёхзначных шестнадцатеричных чисел, разделяя из двоеточиями.

Задача I. Агроном

Представим, что находимся в точке $(0, 0)$. Разрешим себе перемещаться себе на m клеток по одной оси и на k клеток по другой оси. Такое перемещение будем называть ходом (m, k) -коня.

Пример: из точки $(0, 0)$ можно попасть в 8 точек $(1, 2)$ -конём:

$(1, 2)$
 $(2, 1)$
 $(-1, 2)$
 $(-2, 1)$
 $(-1, -2)$
 $(-2, -1)$
 $(1, -2)$
 $(2, -1)$

Таким образом, в задаче спрашивают: в какое количество клеток огорода можно попасть ходами (m, k) -коня такими что, $m^2 + k^2 = n$

Лемма 1: из точки $(0, 0)$ ходом (m, k) -коня можно попасть в те и только те клетки (x, y) , для которых выполнено:

- x и y делятся на $\text{НОД}(m, k)$
- если $m / \text{НОД}(m, k)$, $k / \text{НОД}(m, k)$ нечетные, то $x / \text{НОД}(m, k)$ и $y / \text{НОД}(m, k)$ должны быть одинаковой четности

Доказательство Первый пункт леммы легко доказать, так как каждый ход происходит смещение на величину, кратную $\text{НОД}(m, k)$ по каждой компоненте

Для доказательства второго – нужна раскраска узлов, обе координаты которых делятся на $\text{НОД}(m, k)$

Следствие 1: Каждого (m, k) -коня можно свести к $(d, 0)$ или (d, d) -коню, где $d = \text{НОД}(m, k)$, при этом к какому именно – понятно из четностей m/d и k/d .

Лемма 2: $(d1, 0)$ - и $(d2, 0)$ -коня можно свести к $(\text{НОД}(d1, d2), 0)$ -коню

Лемма 3: $(d1, d1)$ - и $(d2, d2)$ -коня можно свести к $(\text{НОД}(d1, d2), \text{НОД}(d1, d2))$ -коню

Будем называть (d, d) -коня – d -диагональным, а $(d, 0)$ коня – d -прямым.

Пусть теперь у нас есть несколько (m_i, k_i) -коней, нам нужно понять, в какие клетки можно попасть, комбинируя их ходы в любом порядке.

Сначала заметим, что благодаря леммам 2 и 3 и следствию из леммы 1, можно весь набор коней свести к не более, чем двум коням:

- диагональному (если хотя бы один конь сводился к нему)
- прямому (если хотя бы один конь сводился к диагональному)

Лемма 4: Для любого n множество (m, k) -ходов таких, что $m^2 + k^2 = n$, сводится не более чем к одному коню.

Доказательство:

Пусть $n = d^2 * x^2 + d^2 * y^2$.

1. Если x, y разной четности, то степень вхождения 2 в n нечётна
2. Если x, y одинаковой четности (оба нечетные), то степень вхождения 2 в n чётна.

Это значит, что по степени вхождения 2 в число n можно сказать конь какого типа соответствует n .

Вернемся к задаче.

В случае, если n соответствует только b -диагональный конь, то множество клеток, которые он может посетить, это объединение множеств:

1. клеток $2b$ -коня
2. клеток $2b$ -коня + смещение (b, b)

Таким образом, задача сводится к поиску клеток в прямоугольнике $[x1; x2] * [y1; y2]$, достижимых d -конём. Ограничения задачи позволяют перебрать все возможные x , кратные d на отрезке $[x1; x2]$, и найти количество подходящих y .

Однако можно придумать формулы, для решения последней задачи и получить решение за $O(1)$

Задача J. Незнайка и кубики

Классическое динамическое программирование с квадратной таблицей. Пусть есть исходная последовательность целых чисел

$$S = s_1, s_2, s_3, \dots, s_n$$

. Обозначим за $T(i, k)$ максимальное количество пар, которые можно удалить из подпоследовательности, начинающейся с позиции i и имеющей длину k ($s_i, s_{i+1}, \dots, s_{i+k-1}$), тогда:

1. $T(i, k) = 0$ при $k < 2$. Очевидно, что нельзя удалить пару из последовательности длины меньше 2.
2. $T(i, 2) = 1$ если s_i и s_{i+1} находятся в отношении делимости, и $T(i, 2) = 0$, если нет. Запуск динамического программирования.
3. Определим частичное решение $G(i, r) = \max(T(i, k) + T(i + k, r - k))$ по всем $1 < k < r - 1$. Разрезаем подпоследовательность длины r на 2 части и выбираем оптимальный разрез.
4. Тогда $T(i, r) = \max(G(i, r), T(i + 1, r - 2) + 1)$ если s_i и s_{i+r-1} в отношении делимости, и $T(i, r) = G(i, r)$, если нет. Если крайние элементы подпоследовательности находятся в отношении делимости, то учитываем и вариант, что они образуют пару.

Для величин $T(i, k)$ запомним, каким способом они получены, и используем эту информацию для восстановления пути получения для $T(1, n)$.

Задача К. Факториалы

Отношение факториалов $\frac{a!}{b!}$ по сути является произведением идущих подряд целых чисел, или, иначе говоря убывающей степенью $\frac{a!}{b!} = a^{a-b}$. Если рассмотреть отношение убывающей степени к факториалу (это будет $c!$) с тем же количеством элементов, то это быстро растущая функция очень быстро превышающая 10^9 . Поэтому решение будет либо с очень небольшой разницей $a - b$, либо $c!$ с существенно большим количеством элементов: $c > a - b$.

Во втором случае возникает проблема – факториал является произведением целых, а среди маленьких целых очень много простых чисел, что приводит к тому, что $\frac{a!}{b!}$ очень быстро перестает делиться на $c!$ нацело. Фактически, в диапазоне до 10^9 и при указанных ограничениях существует всего одно число, у которого достигает 23:

$$527784021 = \frac{46!}{26! \times 23!}$$

У этого же числа разность $a - b$ достигает 20. У всех остальных чисел эта разность сильно меньше. Потому, достаточно перебирать все разности от 1 до 20 для всех возможных a , и проверять на какие факториалы они делятся.

Используя формулу подсчёта вхождения простого числа в разложения факториала $r!$ на простые:

$$k = \left\lfloor \frac{r}{p} \right\rfloor + \left\lfloor \frac{r}{p^2} \right\rfloor + \left\lfloor \frac{r}{p^3} \right\rfloor + \dots$$

можно:

1. Обойтись без длинной арифметики.
2. Сделав предварительные расчёты для $p = 2$, по числам n, a, b определять с точностью до 2.
3. Итеративно двигать правую границу допустимого решения: пусть у нас есть число $p > a/2$, тогда оно входит в разложение $a!$ ровно один раз. Если оно не делит исходное число n , то оно обязано входить в $b!$, так как $b \leq c$, но это значит, что b не меньше чем p . Проверив числа с таким b , можно сдвинуть верхнюю границу поиска к p . Простые числа идут достаточно густо, потому диапазон перебора b будет невелик – нужно только каждый раз выбирать максимально возможное p .

Задача Л. Битовые операции

Задача решается разбором случаев.

Один из вариантов разбора:

1. $n = 1$. В таком случае выводим, например, 3 и 2;

2. n – нечётное. Ответом будут числа $n - 1$ и 1 ;
3. n – чётное. Ответом будут числа $n + 1$ и 1 , однако случай $n = 10^{18}$ можно рассмотреть отдельно.
4. При $n = 10^{18}$ можно было найти самый младший единичный бит (он имеет номер 16) и вывести числа $10^{18} - 2^{16}$ и 2^{16}

Также можно разделить множество чисел на те, которые являются степенью двойки, и те, которые степенью двойки не являются. Если n – степень двойки, выводим $n + 1$ и 1 , в противном случае находим самый младший единичный бит (пусть он имеет номер k), удаляем его из числа и выводим $n - 2^k$ и 2^k . При этом отдельно рассматриваются случаи $n = 1$ и $n = 0$.

Задача М. Менеджер памяти

Для каждого элемента будем поддерживать информацию о ближайших свободных элементах с обеих сторон. Для этого можно использовать нагруженный Disjoin set (оптимально по производительности) или воспользоваться стандартными структурами типа map или set (нужна определенная аккуратность в реализации), также допустимы реализации, использующие дерево отрезков или дерево Фенвика. При выделении элемента нужно актуализировать структуры.

У Disjoin set в каждом элементе достаточно хранить номер ближайшего свободного слева и номер ближайшего свободного справа, модифицировав соответствующим образом операцию слияния корневых деревьев. При выделении свободного элемента проверяем занят ли элемент слева от него – если нет, то делаем слияние с левым соседом. Аналогично справа.